

Foundations of All Concepts Implemented in This Project

Nicolas Becker

September 10, 2026

Contents

1	Abstract	2
2	Introduction	2
2.1	GAN Issues	2
3	Architectures	2
3.1	Vanilla GAN	2
3.2	DCGAN	3
3.3	Resnet-GAN	3
3.4	Conditional GAN	3
3.5	Super Resolution GAN	3
3.6	BigGAN	3
3.7	Summary and Comparison	3
4	Objective Functions	3
4.1	Least Square GAN	3
4.2	Wasserstein GAN	4
4.3	Wasserstein GAN with Gradient Penalty	4
4.4	Hinge GAN	4
4.5	Summary and Comparison	4
5	Regularizers	4
5.1	Feature Matching	4
5.2	Minibatch-Discrimination	4
5.3	Spectral Normalization	5
5.4	Initializing Techniques	5
5.5	Summary and Comparison	5
6	Experiments	5
6.1	Methods	5
7	Combinations tried	5
7.1	Limitations	5

1 Abstract

This project will cover several GAN

2 Introduction

In 2014 a certain type of generative models was invented by **goodfellow2014generative**, these new type of neural networks are called *Generative Adversarial Networks* (GANs). Compared to new generative models like **DALL-E** developed by Open-AI, which is based on *Diffusion Models* (These models will probably come also in a own project as soon as GANs are accomplished), their ability of creating diverse and realistic looking images is limited. This comes with the cost of relevance in modern applications. However, GANs are still used in a wide range of applications from data augmentation and self supervised learning. One of the biggest advantage still remains in the fast inference compared to the diffusion models.

2.1 GAN Issues

Training GANs is inherently complex firstly due to their non convergence nature. This is easily seen by equation 1, their objective is against each other. Therefore the training is instable and may not converge til a global minima.

Secondly, due to their ability to be stuck in a local minima and only produce limited modes of the actual training data. The generator is only producing partiell parts of the training data. For instance, imagine that the generator is only producing the number 6 and 7 but not the others. Unfortunately, if the discriminator is not well trained only some nodes are sufficiently to fool the discriminator. This is called Mode colapse.

3 Architectures

3.1 Vanilla GAN

The beginning of this hopefully reasonably overview, will be the start of GANs with Goodfellow et al. [1], which might be the most influential paper about GANs. Shortly said, GANs consist of two neural networks working against each other to surpass the performance of the counterpart. One of the participants is the Generator that tries to generate data that resamples the original data. This procedure consists on the assumption that each dataset can be seen as a probability distribution. Following, this assumption the generator tries to learn the distribution of the dataset and can sample realistic looking images from the learned distribution. The Discriminators' objective is to distinguish whether real or generated data (fake data) was fed into the discriminator. The Generator learns to fool the discriminator, while the discriminator has to learn to distinguish. Mathematically, this can be seen with expression:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]. \quad (1)$$

the objective of the discriminator is actually to maximize the probability of whether beeing real in the first term or zero in the second term.

3.2 DCGAN

Based on the equations like (1), DCGAN is equivalent in the computation but differ only in the architecture. DCGAN stands for *Deep Convolutional GAN*, better said they are based on convolutional layers, thereby the ability to resample photo realistic images is increased. The invention of DCGAN was made by Radford, Metz, and Chintala [3]. Additionally, they found good hyperparameters to avoid instable training. New opportunities came across due to their convolutional nature. For instance, it became possible to use interpolation within the latent space, which makes it possible to linearly interpolate styles across a dataset. The main application from the paper invented interpolation with celebA to change semantic features of the faces.

Since convolutional layers introduce an inductive bias while processing images the quality of the generated images increases. Unfortunately, the quality is still limited and the training process is also instable.

3.3 Resnet-GAN

Training deep networks is heavy, but with the rise of resnet, the ability of training became more possible. The speciality of resnet is the skip connection between the input of the layer and the output. Implementarily, the input is added to the output, so that the model only has to learn the difference, thus the gradient flow is more fluently.

3.4 Conditional GAN

3.5 Super Resolution GAN

3.6 BigGAN

3.7 Summary and Comparison

4 Objective Functions

4.1 Least Square GAN

In advance to to common GAN loss function lsgan tries to avoid the vanishing gradient problem occuring due to a lack of information retained by the discriminator. In other words if the discriminator becomes to good the gradients aren't useful for training. Since this lies in the nature of minimizing the Jensen-Shanon-Divergence, the authors argued that minimizing the pearson x^2 divergence, provide more plausible gradients for training. Invented was lsgan in 2017 by Mao et al. [2]. The authors claim a more stable training and generated images of higher quality. In figure 1 the issue of vanishing gradients is visualized. While the least squares shows a steady with values unlike zero the BCE shows clearly a loss of information in the positive section of the x-axis.

Accordingly, the loss function is adapted with the following equation:

$$\mathcal{L}_D = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \left[(D(x) - 1)^2 \right] + \frac{1}{2} \mathbb{E}_{z \sim p_z} \left[D(G(z))^2 \right] \quad (2)$$

$$\mathcal{L}_G = \frac{1}{2} \mathbb{E}_{z \sim p_z} \left[(D(G(z)) - 1)^2 \right] \quad (3)$$

Unlike the BCE loss where samples that are correctly classified lose gradient information lsgan

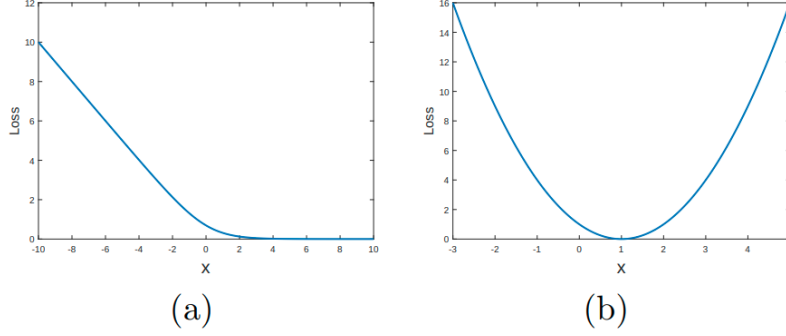


Figure 1: Comparison of vanilla Binary Cross Entropy and Least Squares [2].

penalize even those samples. Moreover, lsgan pushes the samples toward the decision boundary, thus the generated samples lies near the manifold of real data. Additionally, the penalizing generates more gradient information even though for samples that are already correct classified [2].

4.2 Wasserstein GAN

4.3 Wasserstein GAN with Gradient Penalty

4.4 Hinge GAN

4.5 Summary and Comparison

5 Regularizers

5.1 Feature Matching

The idea is to condition the discriminator in rather distinguishing between real and fake data to match the features of a intermediate layer of the discriminator for real and fake data. This is implemented by changing the objective of the generator in matching the statistics of e.g. the last convolutional features from the real and the fake data. The mean of the features is taken and with L2-norm compared. The authors argue that empirically this provides better training mechanics. In particular, overtraining the generator is addressed. It is not proven to practically find a fixed point where the generator matches perfectly the training data. Whereas the fixed point should exists like in usual GAN training. However, the authors argue that feature matching is surprisingly effective in training GANs especially if the training is unstable [4].

5.2 Minibatch-Discrimination

Another stabilizing technique proposed by Salimans et al. [4] is addressing the issue of mode collapsing, where the generator only produces samples that are almost identical. In conclusion, the generated images only resample a certain mode of the training data. The gradients of the discriminator points for different samples in a similar direction, which is believed to be correct by the discriminator. But the samples are classified independently, which lead to no coordination of gradient directions. To solve this Salimans et al. [4] proposed to rather look at a collection of images instead of just one by one.

Essentially, for each image in the batch a learnable representation is learned, which allows to compute a similarity between each sample in the batch. Afterwards, the similarity is multiplied by an exponential function to keep similar samples close to zero and vice versa. Mini batch

discrimination can be seen as a new layer in the discriminator, which computes the similarity and concatenate this information with an intermediate layer for further processing. However, the discriminator outputs still a bool for real or false. For semi supervised learning this procedure showed best performance, with obtaining the best possible discriminator. The authors claim that this procedure is superior to feature matching because it quickly generates realistic appealing images [4].

5.3 Spectral Normalization

5.4 Initializing Techniques

5.5 Summary and Comparison

6 Experiments

6.1 Methods

7 Combinations tried

An exemplary overview of the structure that will follow soon, some trainings are in progress.

Table 1: Overview of the evaluated GAN configurations.

ID	Architecture	Loss	Regularization
A	DCGAN	Vanilla GAN	None
B	DCGAN	LSGAN	Spectral Normalization
C	DCGAN	LSGAN + Feature Matching	Spectral Normalization
D	DCGAN	LSGAN	Minibatch Discrimination
E	DCGAN	LSGAN	Label Smoothing
F	DCGAN	LSGAN	Spectral Normalization + Label Smoothing
G	DCGAN	LSGAN	Spectral Normalization + Minibatch Discrimination
H	DCGAN	WGAN	Weight Clipping
I	DCGAN	WGAN-GP	Gradient Penalty

7.1 Limitations

Unfortunately, training GANs is highly time and computation consuming. Therefore not all trainings are done until optimum. The trainings computed here are invariably done on one Nvidia RTX 4090. Trainings that are taking several days are mostly partially splitted over a few days. However, even the comparison is quite complicated, because the metrics are biased to the dataset, therefore some of the metrics are only comparable on the same dataset and are not transferable. This is why I mostly used CelebA. Also a comparison of higher resolutions might be a future thing here due to the exponential growing computational cost. Whereas it might be pretty interesting to observe the quality and impact of all modules on higher resolutions. But this brings us to the next project (future work diffusion models, to overcome a low resolution, but with the cost of extremely more computational power) Diffusion models. Up to now everything here might be obsolete, because it is an ongoing project, so don't expect much of it.

References

- [1] Ian J. Goodfellow et al. “Generative Adversarial Nets”. In: (2014). arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [2] Xudong Mao et al. “Least Squares Generative Adversarial Networks”. In: (Nov. 2016). DOI: 10.48550/ARXIV.1611.04076. arXiv: 1611.04076 [cs.CV].
- [3] Alec Radford, Luke Metz, and Soumith Chintala. “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. In: (2016). arXiv: 1511.06434 [cs.LG]. URL: <https://arxiv.org/abs/1511.06434>.
- [4] Tim Salimans et al. “Improved Techniques for Training GANs”. In: (June 2016). DOI: 10.48550/ARXIV.1606.03498. arXiv: 1606.03498 [cs.LG].